

制約付きメモリ環境における SQIsign v2/v3 の省メモリ実装

lifetime-aware scheduling と exact replay 付き norm sketch

Hiro Nakanishi

独立研究者

quantumsity@protonmail.com

プレプリント版：2026 年 9 月 4 日

概要

SQIsign は、同種写像と四元数算術に基づく耐量子署名方式であり、公開鍵と署名が非常に小さい。しかし、通信表現の小ささは実装時の作業メモリの小ささを意味しない。SQIsign v2 の鍵生成と署名では、多数の固定精度多倍長整数、格子状態、候補列が同時に生存する。実際、出発点とした compact 実装では、一回の `find_uv` が保持する五つの確保領域だけで 6,339,868 bytes を要し、RP2350 のオンチップ SRAM 532,480 bytes の 11.91 倍に達する。2026 年 9 月に公開された SQIsign v3 は、固定精度整数、格子簡約、ideal-to-isogeny 変換を再設計し、この v2 固有のボトルネックを除去した。本稿は、この版変更を手法の転用可能性を調べる独立した追試として扱う。

本稿では、SQIsign v2.0.1 Level I の KeyGen、Sign、Verify を、Raspberry Pi Pico 2 の RP2350 (Arm Cortex-M33) 上で、GMP、動的メモリ確保、外部 PSRAM を用いずに実行するための二つの手法を示す。第一に、作業オブジェクトを型、サイズ、アラインメント、生存区間で表し、同時生存しない領域を一つの呼出側所有アリーナへ重畳する lifetime-aware arena scheduling を用いる。第二に、候補ノルムの全多倍長表を常駐させず、厳密なビット長と上位 64 bit のみを保持する certified norm sketch を導入する。sketch が同値の場合は、保持済み候補ベクトルから両ノルムを再生成して全精度比較する。この exact replay により、近似や collision 確率を仮定せず、元の全精度比較と同一の候補順序を保つ。以下、この二手法を統合した構成を v2 提案実装と呼ぶ。

Level I/RADIX32 実装では、完全な操作アリーナを 353,008 bytes から 172,080 bytes へ 180,928 bytes (51.2532%) 削減した。実機 firmware の排他的 SRAM 予約量は 321,408 bytes となり、211,072 bytes を未予約として残した。flash 増加は 880 bytes である。12 個の凍結決定的トランスクリプトは byte 単位で一致した。さらに、保存済み NIST-v2 Level-I の公式 100-vector request を用いる専用 KAT を独立に二回完走し、各回 100 本で KeyGen、Sign、Open、改変拒否、arena 境界・消去を確認した。二つの低メモリ応答は同一 commit の通常 D13 API 応答と byte 単位で一致した。一回の実機 boot でも KeyGen、Sign、Verify を順に完了した。実測時間はそれぞれ 2,698.150029 s、7,611.258527 s、0.814341 s であり、鍵生成と署名はなお実用速度に達していない。

SQIsign v3.0 の追試では、公式 p324_3/m4f 実装と、一関数内の二組の非交差生存区間を重畳した v3 適応実装を別ディレクトリで構築した。v3 適応実装は、署名時の PSP 上書き深さを 101,060 bytes から 97,132 bytes へ 3,928 bytes (3.8868%) 削減した。KeyGen と Verify の上書き深さは変化しなかった。ホスト試験に加え、RP2350 上で順序を反転した 5 組、計 10 回の公式既知解テストが全て合格し、stack 値も各版内で一定だった。署名時間の対ごとの増加率は中央値 0.4182% (範囲 0.4101–0.4226%) だったが、単一基板・単一ベクトルに限られる。また、v3 適応実装には可変長 stack 記録が残るため、v2 提案実装と同じ静的リンク閉包を達成したとは主張しない。v2 提案実装の固定対ランダム timing screen と全署名の制御フロー・address 検査は入力依存性を検出した。従って、本稿は二世代にわたるメモリ設計と機能同値性を示すが、定数時間性、電力・電磁波耐性、製品利用可能性を主張しない。

キーワード: 耐量子暗号、SQIsign、マイクロコントローラ、省メモリ実装、ライフタイム解析、サイドチャネル

1 はじめに

NIST の追加耐量子デジタル署名標準化では、SQIsign が第 3 ラウンド候補の一つに選ばれている [1, 2]。SQIsign は、超特異楕円曲線間の同種写像と四元数イデアルの対応を利用し、非常に小さな公開鍵と署名を実現する [3, 4, 5]。この通信上の compactness は、帯域、不揮発メモリ、証明書サイズが制約される機器にとって重要である。

しかし、wire format の小ささと、鍵生成・署名時の working set の小ささは別の性質である。SQIsign の署名側処理では、四元数算術、4 次元格子処理、ノルム方程式、候補探索、多倍長整数演算が重なる。v2 公式実装は GMP と動的確保を前提とし、固定精度整数版 [6] と Compact Quaternion Algorithms[7] は整数値の上界を与えたものの、プロトコル全体で同時に生存する配列の配置までは解決しない。実際、本研究の出発点とした compact 実装では、一回の `find_uv` が保持する五つの確保領域だけで 6,339,868 bytes を要する。これは対象とする RP2350 の全オンチップ SRAM 532,480 bytes の約 11.91 倍である。固定精度化によって heap を除去しても、同じ領域を巨大な stack または静的配列へ移すだけでは Cortex-M 上で実行できない。

組込み SQIsign については、pqm4 の追加署名候補調査が GMP と動的確保を理由に v2 署名側の統合を見送り [8]、後続研究も主として Verify を対象としてきた [9, 10]。完全な v2 KeyGen、Sign、Verify を扱う高速実装は通常の host または Armv8-A 環境を対象とする [11]。一方、2026 年 9 月 1 日に公開された v3.0 は、外部 GMP と浮動小数点ライブラリへの依存を除去し、Cortex-M4 向けの完全な KeyGen、Sign、Verify 実装を提供した [5, 12]。この変更により、v2 のメモリ実現可能性と、再設計後の v3 へ同じ原理を転用できるかは、分けて評価する必要がある。

本研究では暗号方式そのものを変更せず、値の表現、保持、再生成、および所有権を再設計する。具体的には、次の四つの研究課題を扱う。

- Q1. SQIsign v2.0.1 の実 KeyGen、Sign、Verify を、GMP、heap、外部 PSRAM なしで Cortex-M 級のオンチップ SRAM 内に収められるか。
- Q2. 全精度の候補ノルム表を常駐させず、元実装と厳密に同じ比較順序と暗号出力を保てるか。
- Q3. RAM 削減を stack、flash、実行時間へ転嫁していないかを測定し、安全性上の新旧の入力依存性を区別できるか。
- Q4. v2 固有の候補探索が消えた SQIsign v3 でも、生存区間に基づく領域重畳が別の支配的 stack frame へ転用できるか。

以下では、certified norm sketch 導入直前の静的アリーナ構成を**全精度表実装**、これに sketch と exact replay を加えた最終構成を *v2 提案実装* と呼ぶ。また、公式 v3.0 へ局所的な領域重畳だけを加えた構成を *v3 適応実装* と呼ぶ。

本稿の貢献は以下の通りである。

1. **プロトコル全体の lifetime scheduling.** 作業領域を型、サイズ、アラインメント、生存区間、秘密性属性を持つオブジェクトとして表し、干渉しない領域を型付き arena へ重畳する問題として定式化する。KeyGen、Sign、Verify を逐次実行する operation owner まで含め、heap だけでなく同時生存 SRAM 全体を削減対象とする。
2. **exact replay 付き certified norm sketch.** 候補ノルムのビット長と上位 64 bit を順序証明子として保持し、証明子が同値の場合だけ保持済み候補から全ノルムを厳密再生成する。比較器が元の全精度比較と同一の全順序を与えることを示し、意図的 collision fixture を含む差分試験で検証する。
3. **静的な Cortex-M 実装と Pareto 評価.** SQIsign v2.0.1 Level I/RADIX32 について、GMP、allocator、VLA を選択済み link closure から除去した。v2 提案実装は operation arena を 353,008 bytes から 172,080 bytes へ 51.2532% 削減し、排他的 SRAM 予約を 321,408 bytes、未予約 SRAM を 211,072 bytes とする。flash 増加は 880 bytes である。12 個の凍結トランスクリプトに加え、公式 request を用いる 100-vector KAT

を独立に二回完走し、同一 commit の通常 API との byte 一致を保ったまま RP2350 実機で KeyGen、Sign、Verify を一回の boot 内に完了する。

4. **SQIsign v3 への転用追試.** 公式 v3.0 と v3 適応実装を、v2 および相互から分離した source、build、result ディレクトリで評価する。v3 適応実装は `quat_111_dual_reduce_ideal` 内の二組の非交差生存区間を重畳し、公式既知解テストとの一致を保ったまま Sign の PSP 上書き深さを 3,928 bytes 削減する。v2 固有の norm sketch は転用せず、一般化できた要素とできなかった要素を分離する。
5. **安全性を含む claim boundary.** 固定対ランダム of timing screen、全 Sign の edge/address 差分、および既報 SPA[13] との経路対応を評価する。現実装に入力依存性が残るという positive result を報告し、省メモリ化を定数時間化または電力・電磁波耐性として主張しない。

本稿の構成は次の通りである。節 2 で v2 と v3 の相違、プラットフォーム、脅威モデルを整理し、節 3 で先行実装と本研究の位置付けを明確にする。節 4 で同時生存メモリ最小化を定式化し、節 5 and 6 で v2 の arena scheduling と certified norm sketch、および v3 への局所的な転用を述べる。節 7 and 8 で版ごとに正しさ、SRAM、stack、flash、実行時間を評価し、節 9 でサイドチャンネル上の限界を検証する。最後に節 10 and 11 で一般化可能性と妥当性への脅威を議論する。

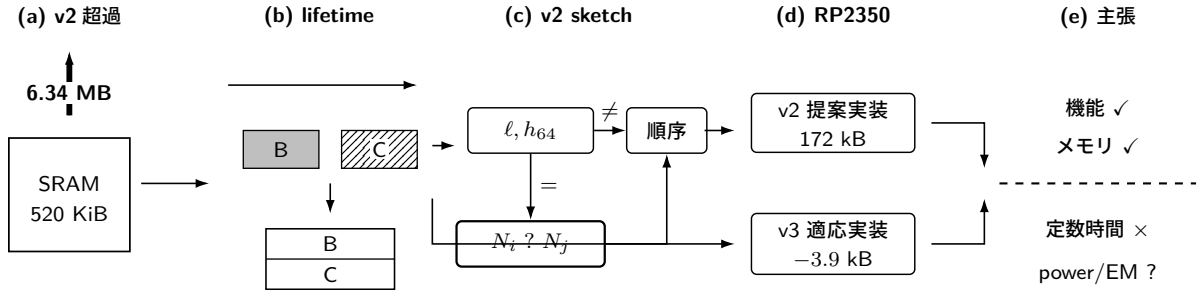


図 1 本研究の全体像。v2 提案実装では lifetime scheduling と certified norm sketch を組み合わせ、v3 適応実装では lifetime scheduling だけを別の格子簡約領域へ転用する。両系列を RP2350 上で評価し、安全性の主張範囲を機能・メモリ評価から分離する。

2 背景

2.1 SQIsign v2 と実装上の特徴

SQIsign は、超特異楕円曲線の自己準同型環と四元数代数の極大整環の対応を計算に用いる署名方式である [3]。本研究が主対象とする v2.0.1 は、署名の応答計算で楕円曲線の積、すなわち二次元アーベル多様体上の (2,2)-isogeny chain を用いる第 2 ラウンド仕様系列である [4]。従って、本稿の v2 実験は「一次元 SQIsign 実装」ではなく、旧 Round-1 実装や一次元検証実装のメモリ・性能値をそのまま比較対象にはできない。v2 提案実装は Level I の完全な KeyGen、Sign、Verify 経路を対象とする。

計算の主要部分は、有限体・楕円曲線演算、四元数イデアル演算、4 次元格子処理、および固定精度多倍長整数演算からなる。有限体側はコンパイル時に幅が定まる一方、四元数側では大きな中間整数、候補列、ソート用情報、格子状態が同時に生存し得る。固定精度化は動的な整数長を排除するが、全ルーチンへ一様な最大幅を適用すると大きな配列が stack または静的領域へ移るだけであり、総 SRAM が小さくなるとは限らない。

2.2 SQIsign v3 による設計変更

SQIsign v3.0 は 2026 年 9 月 1 日に公開された第 3 ラウンド仕様である [5]。Level I に相当する p324_3 では、公開鍵、秘密鍵、署名がそれぞれ 83、270、200 bytes である。v3 は、新しい攻撃評価に対応してパラメータを拡大する一方、四元数イデアルの inert representation、整数成長量の厳しい上界、新しい格子簡約、ideal-to-isogeny

変換、および応答分布を導入した。外部 GMP と DPE 浮動小数点ライブラリは、独自の固定精度整数実装と浮動小数点を使わない処理へ置き換えられた。公式 source は x86_64、Arm64、Cortex-M4 を対象とし、Cortex-M4 について完全な KeyGen、Sign、Verify の性能と stack 使用量を報告する [5, 12]。

v3 は v2 への小さな修正ではない。v2 提案実装の最大削減を生んだ `find_uv` と全精度候補ノルム表は、公式 v3 source commit 6d017708db403bf83977fa70770fc4f7f9e9ff21 の到達経路に存在しない [12]。このため、certified norm sketch を v3 へ移植する対象はない。一方、生存区間が非交差する大きな局所状態は v3 にもあり、lifetime scheduling の対象になり得る。本稿はこの差を利用し、v2 固有のデータ縮約と、版を越えて使える配置原理を実験的に分離する。なお、v3 仕様は応答分布の変更により v2 と v3 の安全性仮定が技術的には直接比較できないと説明している [5]。本稿も暗号学的安全性の版間順位を主張しない。

2.3 対象プラットフォーム

対象は Raspberry Pi Pico 2 に搭載された RP2350 である。実験では Arm Cortex-M33 を 1 core、system clock を 150 MHz とし、オンチップ SRAM 532,480 bytes のみを使用した。外部 PSRAM は使用せず、コードおよび読み出し専用データは内蔵 flash から実行・参照する。最終イメージは Pico SDK 2.3.0、Arm GNU Toolchain 15.2.1、Release 構成でビルドした。

RP2350 は一般的な Cortex-M4 評価ボードより SRAM が大きい、SQIsign の元の候補ワークスペースはなお桁違いに大きい。また、割込み用 MSP、アプリケーション用 PSP、グローバルな操作所有者、ランタイムデータをすべて同じ物理 SRAM 内で共存させる必要がある。このため、本稿では「heap 使用量」ではなく、リンク時予約、stack 予約、および操作アリーナを含む排他的 SRAM 予約量を評価指標とする。

2.4 脅威モデルと主張範囲

機能・メモリ評価では、境界外アクセス、workspace guard 破壊、未消去、allocator/GMP への意図しない依存、ホスト参照との出力不一致を失敗とする。物理攻撃者については、署名中の実行時間、制御フロー、メモリアドレス、消費電力、電磁波を観測できる強い実装攻撃者を想定する。ただし、本稿で取得済みなのはホスト上のタイミング・構造トレースと RP2350 上の局所タイミング診断であり、完全署名の電力・電磁波波形は未取得である。

したがって、本稿の成功条件は「機能経路がメモリ制約内で再現可能に完走すること」である。秘密非依存の実行、マスキング、フォールト耐性、製品品質の乱数生成は成功条件に含めず、未達の安全性要件として明示する。

3 関連研究

lifetime に基づく領域重畳そのものは新しい原理ではない。同時生存する値を干渉辺で表す考え方は graph-coloring 型 register allocation で確立され [14]、異なる大きさの配列や構造体を対象とする compile-time memory allocation も研究されている [15]。従って本稿は、arena 重畳一般の発明を主張しない。本稿固有の対象は、SQIsign の完全な K/S/V 閉包について、分岐・retry・失敗経路、型と alignment、秘密消去まで含む生存契約を作り、exact replay による resident state 削減と組み合わせ、最終 ELF と実機で総 SRAM を閉じることである。

固定精度整数版 SQIsign は、四元数中間値へ Level I/III/V で 7026/10713/14150 bit の一様上界を与え、GMP を使わない実装可能性を示した [6]。Compact Quaternion Algorithms の現行改訂は、この上界を 1665/2521/3319 bit へ縮小し、host 上の鍵生成・署名を大幅に高速化した [7]。本研究の凍結実装も Level I で 1665-bit magnitude bound を用いる。本研究はこの値幅の結果を出発点とするが、同時生存する候補表とプロトコル全体の領域配置を主対象とする。

Qlapoti は、四元数イデアルを同種写像へ変換する IdealToIsogeny 処理を改善し、SQIsign の速度と heap 使用を大きく削減した [16, 17]。その後の Qlapoty は、公開済み Qlapoti 記述と実装の差異および失敗確率解析を再検討し、新しい norm-equation algorithm と SQIsign 署名の追加高速化を報告した [18]。これらは本稿の速度改善に重要な入力である。一方、報告された heap 値または host 時間は、stack、割込み予約、mutable globals を含む

Cortex-M の排他的 SRAM 予約ではなく、本稿の最終実装と同一条件の直接比較ではない。

pqm4 による追加署名候補の調査では、SQIsign は GMP と動的確保のため組み込み統合から除外された [8]。その後、Cortex-M4 向け高速有限体演算の生成により v2 Level I Verify が高速化されたが、KeyGen と Sign は対象外である [9]。一次元 SQIsign の Cortex-M4 実装も主な実機対象は Verify である [10]。一方、Armv8-A NEON 向け v2 実装は完全 K/S/V を高速化するが、64-bit SIMD、GMP、より大きな memory system を前提とする [11]。

公式 v3 実装は、この実装状況を変えた。v3 は独自の固定精度整数層を採用し、Cortex-M4 上の完全 K/S/V と、Level I で KeyGen、Sign、Verify の stack 使用量 56、99、37 kB を報告する [5, 12]。従って、本稿の v3 追試は「初めて Cortex-M で K/S/V を動作させる」という主張ではない。公式 v3 を無改変の対照群として固定し、同じ source commit と実装条件のまま、追加の lifetime scheduling が署名 stack を削減するかを調べる対照実験である。

安全性については、整数算術 [19]、4 次元格子簡約 [20]、四元数 routine [21] ごとの定数時間化が進んでいる。しかし、いずれも現在の完全な低メモリ signer 全体の定数時間性を与えるものではない。既報 SPA [13] は、この空白を実装上の現実的な問題として示している。

表 1 本研究に直接関係する実装研究の位置付け

研究	完全 K/S/V	対象	本稿との差
Official v2[4]	host では有	x86/GMP	heap・GMP 前提
Official v3[5]	有	Cortex-M4 を含む	固定精度化済み。本稿 v3 追試の対照群
Fixed precision[6]	host	x86	値幅を証明、全同時生存 SRAM は未報告
Compact quaternion[7]	host	x86	本稿の出発点、公開実装の候補表が MCU SRAM 超過
Qlapoti/Qlapoty[16, 18]	host の Sign 経路	x86/GMP	algorithm・heap・速度を改善、stack 込み MCU 実測ではない
pqm4 調査 [8]	組み込み統合から除外	Cortex-M4	GMP・動的確保が障壁
m4-modarith[9]	Verify のみ	Cortex-M4	現行 v2 の組み込み K/S なし
SQIsign on ARM[11]	有	Armv8-A/NEON	Cortex-M 級でなく GMP を保持
本研究	有	RP2350/M33	v2 提案実装は静的閉包、v3 適応実装は局所 stack 重畳

2026 年 9 月 3 日までの公開文献・source 監査では、v2.0.1 Level I の実 KeyGen、Sign、Verify を、Cortex-M 級実機、オンチップ SRAM のみ、GMP なし、動的確保なし、全操作の測定済み SRAM 520 KiB 未満という条件で同時に示す先行例は見つからなかった。この検索結果は v2 に限定される。v3 については公式実装が既に Cortex-M4 の完全 K/S/V を提供するため、同じ不在主張を行わない。いずれも日付付き検索結果であり、「世界初」の証明ではない。投稿直前に仕様、ePrint、公開 repository を再監査する。

4 問題設定：同時生存メモリの最小化

4.1 型付きオブジェクトと生存区間

操作中に使用するオブジェクト集合を $O = \{o_1, \dots, o_n\}$ 、入力、retry、失敗 return を含む許容実行トレースの集合を $\mathcal{T}$ とする。各 o_i は、サイズ $s_i \in \mathbb{N}_{>0}$ 、アラインメント $a_i \in \mathbb{N}_{>0}$ 、型 τ_i 、秘密性属性 σ_i を持つ。トレース $t \in \mathcal{T}$ において o_i が生存する時点の集合を $L_i(t)$ とし、そのトレースに現れない場合は $L_i(t) = \emptyset$ とする。干渉辺を

$$(o_i, o_j) \in E \iff \exists t \in \mathcal{T} : L_i(t) \cap L_j(t) \neq \emptyset \quad (1)$$

で定義し、このグラフを $\mathcal{G} = (O, E)$ と書く。すなわち、どれか一つの許容経路で同時生存し得る二領域は干渉する。単一の正常系トレースで非交差だったことだけでは、重畳を許さない。

アリーナ内の offset を $x_i \in \mathbb{Z}_{\geq 0}$ とすると、配置は次を満たさなければならない。

$$x_i \equiv 0 \pmod{a_i}, \quad (2)$$

$$(o_i, o_j) \in E \implies (x_i + s_i \leq x_j) \vee (x_j + s_j \leq x_i). \quad (3)$$

境界 guard などを除くアリーナ payload の高水位は

$$M_{\text{arena}} = \max_i (x_i + s_i) \quad (4)$$

である。最小化問題は M_{arena} を目的関数とする alignment 付き storage allocation である。ただし、v2 提案実装は一般の最適配置を解く solver ではない。手作業で定めた phase 分割から実行可能解を C の `struct/union` として構成し、`sizeof`、`offsetof`、`_Static_assert` でその配置を検査する。ここではアリーナの基底 address も各 member の要求 alignment を満たすものとする。実装では byte 配列へ任意に cast せず、型付き union の alignment をコンパイラに任せ、ABI probe で確認する。

M_{arena} は firmware 全体の SRAM 量ではない。前後 guard を $g_{\text{pre}}, g_{\text{post}}$ 、PSP と MSP の予約を $R_{\text{PSP}}, R_{\text{MSP}}$ 、owner と stack 以外の排他的な mutable 領域を R_{other} とすると、本稿の最終指標は

$$R_{\text{excl}} = (M_{\text{arena}} + g_{\text{pre}} + g_{\text{post}}) + R_{\text{PSP}} + R_{\text{MSP}} + R_{\text{other}}. \quad (5)$$

v2 提案実装では、 $M_{\text{arena}} = 172,080$ 、guard 合計 128、 $R_{\text{PSP}} = 131,072$ 、 $R_{\text{MSP}} = 8,192$ 、 $R_{\text{other}} = 9,936$ であり、 $R_{\text{excl}} = 321,408$ bytes となる。read-only な XIP 領域と未予約 SRAM はこの式に加えない。

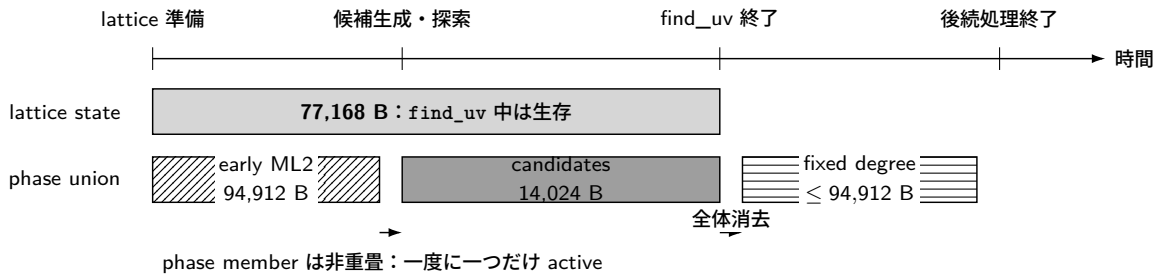
4.2 安全な重畳の条件

領域の重畳は、単に「同時には使わないはず」という経験則では認めない。二つの型付き領域を同一 offset へ配置する条件を次のように定めた。

1. 呼出グラフと phase state machine 上で生存区間が非交差である。
2. 呼出先がポインタを phase 外へ保存せず、返却値も当該領域を参照しない。
3. 次 phase への遷移前に秘密値を消去し、必要なら全領域の zero-check を行う。
4. aliasing とアラインメントを、byte 配列への無制限な cast ではなく型付き union member または検査済み accessor で保証する。
5. failure path を含む全 return edge で、guard、消去、公開状態への非 publication を検査する。

実装では、この条件を ABI size probe、`-Wvla -Walloca -Werror`、ASan/UBSan、canary、全消去 fixture、ELF symbol audit で検査した。これらは完全な静的証明ではないが、手作業の lifetime annotation とバイナリ検査の間に独立な failure gate を設ける。

(a) 論理生存区間



(b) 物理配置



図 2 IdealToIsogeny 経路で共有する v2 提案実装の workspace の生存区間と物理配置。lattice state は `find_uv` 中を通して生存するため、その間の phase member と重畳できない。`find_uv` の共通出口で全体を消去した後は、同じ phase union を fixed-degree 処理が再利用する。図の幅は時間順を表し、実行時間の比率ではない。

4.3 操作所有者の直列化

組込み API では、KeyGen、Sign、Verify が同時に実行されない構成が一般的である。本実装は三操作を一つの *operation owner* の異なる union member として表現し、操作終了時に member と共有アリーナを消去する。これにより、必要 SRAM は三操作の和ではなく最大値となる。ただし、再入可能性、並行署名、複数 core からの同時利用は失われる。本実装は 1 core ・ 逐次実行を明示的な API contract とする。

5 提案手法

5.1 段階的な resident state 削減

出発点の Compact 実装では、候補ベクトル、全ノルム、商、行数、ソート record を一回の `find_uv` で同時に保持していた。本研究では、以下の変換を、各段階でトランスクリプト一致を確認しながら適用した。

1. 値域が $[-2, 2]$ と証明される 4 座標候補を、広い多倍長表現から `int8_t[4]` へ pack する。
2. ソート record の候補コピーをやめ、`uint16_t` の index permutation をソートする。
3. 候補ごとの商を常駐させず、使用地点で決定的に再計算する。
4. 全行を保持する処理を二行 streaming へ変換し、確定した行を再利用する。
5. 大きな VLA を呼出側所有 workspace へ移し、選択済み Arm リンク閉包の動的 stack frame を 0 件にする。
6. ML2、候補探索、theta chain、離散対数、batch inversion、多倍長一時領域を phase ごとに重畳する。
7. KeyGen、Sign、Verify を一つの逐次 operation owner へ統合する。
8. 最後に、全精度ノルム行を certified sketch と exact replay で置換する。

前半の pack、index 化、streaming は resident state そのものを減らし、後半の arena 化は同じ領域を時間方向に再利用する。再計算は flash と時間を増やし得るため、各変換は RAM ・ stack ・ flash ・ 時間の四指標で評価する。

表 2 は、最終点だけでは見えない設計判断を、各変換の内容と計上境界に沿って示す。「二行ストリーミング」までの値は `find_uv` 候補領域であり、それ以降は lattice state を含む caller-owned workspace または operation arena の `sizeof` である。いずれも firmware 全体の peak ではない。「格子状態の明示所有」では VLA 77,168 bytes を workspace へ移すため表示値が増えるが、同時生存総量は減らない。これは、保存クラスの移動をメモリ削減と数えないための意図的な ownership checkpoint である。

表 2 主要な変換後の明示領域サイズ。計上境界の異なる値を firmware 全体の peak として比較していない。

構成	明示領域 [B]	その構成で導入した変換
Compact 出発点	6,339,868	同時生存する五つの heap request payload
候補座標の pack	2,044,252	4-byte 候補座標と packed sort record
索引ソート	1,905,728	record copy を廃し、 <code>uint16_t</code> index permutation を再利用
商の遅延再計算	962,240	候補ごとの商を常駐させず、使用時に再計算
二行ストリーミング	275,840	全 7 行の常駐を二行へ変更
格子状態の明示所有	353,008	77,168 byte の lattice VLA を型付き owner へ移動（同時生存量は不変）
全精度表実装	353,008	ML2、theta、dlog 等を重畳し、選択済み Arm 閉包の動的 frame を除去
v2 提案実装	172,080	full-norm 二行を sketch+exact replay へ置換し、phase 最大 member を ML2 へ移動

5.2 certified norm sketch

候補 v のノルム $N(v)$ は、保持済みの 4 座標 packed vector と、その行について保持している Gram 行列 G_j および調整ノルム d_j から決定的な二次形式評価として再生成できる。この性質を使い、各候補について全精度整数 $N(v)$ を保持する代わりに、次の sketch だけを保持する。 G_j, d_j は秘密由来状態を含み得るため、再生成可能であることをサイドチャネル安全性の根拠には用いない。

定義 1 (上位 w bit sketch). 非負整数 n に対して、ビット長 $\ell(n)$ を $n = 0$ なら 0、 $n > 0$ なら $\lceil \log_2 n \rceil + 1$ とす

る。 $w = 64$ として、

$$h_w(n) = \begin{cases} n, & \ell(n) \leq w, \\ \lfloor n/2^{\ell(n)-w} \rfloor, & \ell(n) > w \end{cases} \quad (6)$$

を定める。組 $S_w(n) = (\ell(n), h_w(n))$ を n の certified sketch と呼ぶ。

比較器は、(1) ビット長、(2) 上位 64 bit、(3) sketch tie 時の再生成済み全精度ノルム、(4) 元の列挙 index、の順に比較する。

命題 1 (順序保存). 各候補 v_i から $N(v_i)$ を決定的かつ厳密に再生成できるとする。上記比較器が与える候補の全順序は、組 $(N(v_i), i)$ を全精度整数比較と *index* 比較で辞書式順序に並べた結果と一致する。

証明. $\ell(N(v_i)) \neq \ell(N(v_j))$ なら、ビット長の小さい非負整数が必ず小さい。ビット長が等しく、 h_{64} が異なる場合、最上位から最初に異なる bit は保持された 64 bit 内に存在するため、その大小は全整数の大小と一致する。両者が等しい場合、sketch だけでは大小を決めず、二つのノルムを厳密に再生成して全精度比較する。ノルムも等しい場合のみ index を比較する。したがって全ての場合で $(N(v_i), i)$ の辞書式順序と一致する。 $\square$

重要なのは、この方法が collision を「十分起こりにくい」と仮定しない点である。例えば 2^{100} と $2^{100} + 1$ はビット長と上位 64 bit が一致するが、必ず exact replay へ進み、正しい順序を返す。したがって暗号学的 hash の collision 確率や、観測コーパスに基づく経験的上界は正しさの仮定に含まれない。

アルゴリズム 1 に、C 実装の `finduv_eval_norm`、`finduv_set_norm_sketch`、`finduv_compare_indices` に対応する処理を示す。EXACTNORM は二次形式値を d_j で割り、余りが 0 かつ商が正であることまで検査する。したがって「丸めた近似ノルム」を exact fallback に使用しない。

5.3 ソートと探索における再計算

ソート時は sketch tie ごとに二つのノルムを一時領域へ再生成し、比較直後に消去する。探索時は、ソート済み index が指す候補から必要なノルムをその場で再生成する。初期 profile では一回の `find_uv` あたり約 7,800 回のソート比較が発生した。最終 attribution corpus では 2,062,361 比較中 5,041 回、すなわち 0.2444% が exact tie であった。探索側の再評価回数は KeyGen fixture で 2–30 回、より広い Sign fixture で 2–578 回であった。これらは性能を説明する観測値であり、入力全体に対する最悪回数の証明ではない。

一行のソートでは、 $m \leq 624$ 個の元 index を持つ `uint16_t` permutation を in-place heapsort する。比較回数は $O(m \log m)$ 、補助領域は $O(1)$ であり、再帰や libc の sort workspace を使用しない。C の比較 callback は error status を直接返せないため、厳密再生成に失敗した場合は `fatal` を latch し、その比較だけは index 順を返す。heapsort 終了後に latch を検査して行全体を拒否するため、fallback 順を正常出力として公開しない。

アルゴリズム 2 to 4 は、一行の生成・整列、二行の厳密探索、二行だけを常駐させながら元の三角形順 (j_1, j_2) を保つ schedule を分けて示す。ADMISSIBLE は、元実装の符号代表、全座標が 2 または 3 の倍数である候補の除外、および該当する基底における位数 4 の対称性代表を同じ走査順で判定する。Level I では $r \leq 7$ 、一行の capacity は 624 である。検証 pass を先に全行へ行うため、早い候補が見つかったことで後方の不正な行を見逃さない。SEARCHEXACT は、二つの候補ノルム n_1, n_2 に対する

$$un_1 + vn_2 = T, \quad u > 0, \quad v > 0 \quad (7)$$

の候補を元実装と同じ nested-loop 順で探す。具体的には、各合同解列について元実装と同じ $0 < v < \lfloor T/n_2 \rfloor$ を走査境界とする。本稿でいう「最初の解」は、全正整数解に関する新たな完全性主張ではなく、この参照走査領域と順序において最初に受理される解を意味する。v2 提案実装からの呼出しでは追加の sum-of-squares 条件は無効である。探索中も n_1, n_2 を packed 候補から再生成し、選ばれた候補だけを外部出力へ写す。この探索は可変回数であり、定数時間手続きではない。

アルゴリズム 1 exact replay 付き certified norm 比較

入力: packed 候補 v 、Gram 行列 G 、非零の調整ノルム d

出力: 成功時は厳密な正ノルム、10-byte sketch、または $(N(v_i), i)$ の比較結果

1: **function** EXACTNORM(v, G, d)

2: $p \leftarrow \text{WIDENTTOIBZ}(v)$

3: $q \leftarrow \text{QUADRATICFORM}(p, G)$

▷ $q = p^T G p$ を固定精度で評価

4: $(n, r) \leftarrow \text{DIVREM}(q, d)$

5: **if** $r \neq 0$ **or** $n \leq 0$ **then**

6: **return** FATAL

7: **end if**

8: **return** n

9: **end function**

10: **function** MAKESKETCH(n)

11: $\ell \leftarrow \lfloor \log_2 n \rfloor + 1$

12: **if** $\ell > \text{UINT16_MAX}$ **then**

13: **return** FATAL

14: **end if**

15: $h \leftarrow n$ **if** $\ell \leq 64$ **else** $\lfloor n/2^{\ell-64} \rfloor$

16: **return** (ℓ, h)

▷ ℓ : 16 bit、 h : 64 bit

17: **end function**

18: **function** COMPARE(i, j, V, S, G, d)

19: **if** $i = j$ **then**

20: **return** 0

21: **else if** $S[i].\ell \neq S[j].\ell$ **then**

22: **return** CMP($S[i].\ell, S[j].\ell$)

23: **else if** $S[i].h \neq S[j].h$ **then**

24: **return** CMP($S[i].h, S[j].h$)

25: **end if**

26: $n_i \leftarrow \text{EXACTNORM}(V[i], G, d)$; $n_j \leftarrow \text{EXACTNORM}(V[j], G, d)$

27: **if** $n_i = \text{FATAL}$ **or** $n_j = \text{FATAL}$ **then**

28: **return** FATAL

29: **else if** $n_i \neq n_j$ **then**

30: **return** CMP(n_i, n_j)

31: **else**

32: **return** CMP(i, j)

▷ 元の列挙 index で全順序化

33: **end if**

34: **end function**

アルゴリズム 2 一行の候補生成と全精度順への整列

入力: 行 j , resident slot, caller-owned workspace W

出力: 全精度 key $(N(v), i)$ で整列した packed 候補行と個数 m , または FATAL

```
1: function PREPAREROW( $j, \text{slot}, W$ )
2:    $m \leftarrow 0$ 
3:   for  $v \in [-2, 2]^4$  を元実装と同じ順序で走査 do
4:     if not ADMISSIBLE( $v, G_j$ ) then
5:       continue
6:     end if
7:      $n \leftarrow \text{EXACTNORM}(v, G_j, d_j)$ 
8:     if  $n = \text{FATAL}$  then
9:       return FATAL
10:    end if
11:    if  $n \bmod 2 = 1$  then
12:      if  $m = 624$  then
13:        return FATAL
14:      end if
15:       $W.V[\text{slot}, m] \leftarrow \text{PACK4XINT8}(v)$ 
16:       $s \leftarrow \text{MAKESKETCH}(n)$ 
17:      if  $s = \text{FATAL}$  then
18:        return FATAL
19:      end if
20:       $W.S[m] \leftarrow s; m \leftarrow m + 1$ 
21:    end if
22:  end for
23:   $P[k] \leftarrow k$  for  $0 \leq k < m$ 
24:   $\text{HEAPSORT}(P, \text{comparator COMPARE})$ 
25:  if 比較 error が latch 済み then
26:    return FATAL
27:  end if
28:  if not APPLYPERMUTATIONBYCYCLES( $W.V[\text{slot}], P$ ) then
29:    return FATAL
30:  end if
31:  return  $m$ 
32: end function
```

アルゴリズム 3 参照走査順を保つソート済み二行の厳密探索

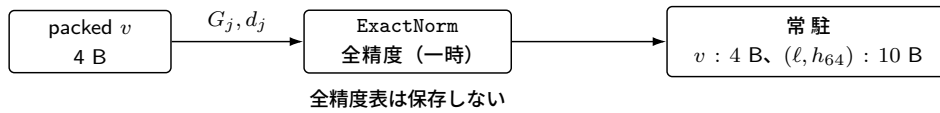
入力: target $T > 0$ 、ソート済み行 $A[0..m_1 - 1]$, $B[0..m_2 - 1]$ 、行記述 $R_k = (G_k, d_k)$ 、対角フラグ δ

出力: 参照実装の走査領域内で equation (7) を満たす最初の $(u, v, i_1, i_2, n_1, n_2)$ 、NOTFOUND、または FATAL

```

1: function SEARCHEXACT( $T, A, B, R_1, R_2, \delta$ )
2:   for  $i_1 \leftarrow 0$  to  $m_1 - 1$  do
3:      $n_1 \leftarrow \text{EXACTNORM}(A[i_1], R_1.G, R_1.d)$ 
4:     if  $n_1 = \text{FATAL}$  then
5:       return FATAL
6:     end if
7:      $a \leftarrow T \bmod n_1$ ;  $s \leftarrow i_1$  if  $\delta$  else 0
8:     for  $i_2 \leftarrow s$  to  $m_2 - 1$  do
9:       if  $\delta$  and  $i_1 = i_2$  then
10:         $n_2 \leftarrow n_1$ 
11:       else
12:         $n_2 \leftarrow \text{EXACTNORM}(B[i_2], R_2.G, R_2.d)$ 
13:        if  $n_2 = \text{FATAL}$  then
14:          return FATAL
15:        end if
16:      end if
17:      if  $n_2^{-1} \bmod n_1$  が存在しない then
18:        continue
19:      end if
20:       $v \leftarrow a n_2^{-1} \bmod n_1$ ;  $q \leftarrow \lfloor T/n_2 \rfloor$ 
21:      while  $v < q$  do
22:         $(u, \rho) \leftarrow \text{DIVREM}(T - v n_2, n_1)$ 
23:        if  $\rho \neq 0$  or  $u \leq 0$  then
24:          return FATAL
25:        end if
26:        if  $v > 0$  then
27:          return  $(u, v, i_1, i_2, n_1, n_2)$ 
28:        end if
29:         $v \leftarrow v + n_1$ 
30:      end while
31:    end for
32:  end for
33:  return NOTFOUND
34: end function
  
```

(a) 生成時



(b) 比較時

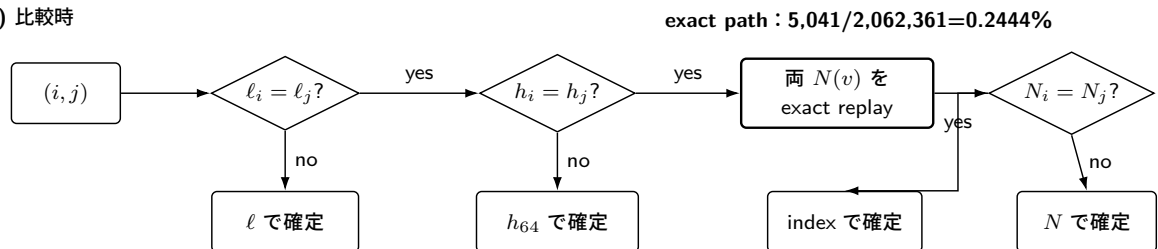


図3 certified norm sketch の保存形式と比較経路。ビット長または上位 64 bit が異なる比較は sketch だけで確定し、両者が一致した場合は packed 候補から全精度ノルムを再生成する。exact replay 率は最終 attribution corpus の観測値であり、正しさはその頻度に依存しない。

アルゴリズム 4 v2 提案実装の検証先行・二行 streaming find_uv

入力: target T 、base ideal と四元数環 $\mathcal{I}$ 、 r 個の order、caller-owned workspace W

出力: 元実装と同じ最初の解、NOTFOUND、または FATAL。全 return で $W = 0$

```
1: function FINDUV-LM( $T, \mathcal{I}, W$ )
2:   if 引数または固定 capacity が不正 then
3:     SECURECLEAR( $W$ );
4:     return FATAL
5:   end if
6:   SECURECLEAR( $W$ )
7:    $L \leftarrow \text{PREPARELATTICESTATE}(T, \mathcal{I}, W.\text{phase.lattice\_mul})$ 
8:   if  $L = \text{FATAL}$  then
9:     return FINISH(FATAL,  $W$ )
10:  end if
11:   $R_j \leftarrow (L.\text{gram}[j], L.\text{adjusted\_norm}[j])$  for  $0 \leq j < r$ 
12:  SECURECLEAR( $W.\text{phase}$ )
13:  for  $j \leftarrow 0$  to  $r - 1$  do
14:     $c[j] \leftarrow \text{PREPAREROW}(j, 0, W)$ 
15:    if  $c[j] = \text{FATAL}$  then
16:      return FINISH(FATAL,  $W$ )
17:    end if
18:  end for
19:  for  $j_1 \leftarrow 0$  to  $r - 1$  do
20:     $m_1 \leftarrow \text{PREPAREROW}(j_1, 0, W)$ 
21:    if  $m_1 \neq c[j_1]$  then
22:      return FINISH(FATAL,  $W$ )
23:    end if
24:    for  $j_2 \leftarrow j_1$  to  $r - 1$  do
25:      if  $j_2 = j_1$  then
26:         $(V_2, m_2) \leftarrow (W.V[0], m_1)$ 
27:      else
28:         $m_2 \leftarrow \text{PREPAREROW}(j_2, 1, W)$ ;  $V_2 \leftarrow W.V[1]$ 
29:        if  $m_2 \neq c[j_2]$  then
30:          return FINISH(FATAL,  $W$ )
31:        end if
32:      end if
33:       $V_1 \leftarrow W.V[0][0..m_1 - 1]$ 
34:       $z \leftarrow \text{SEARCHEXACT}(T, V_1, V_2, R_{j_1}, R_{j_2}, j_1 = j_2)$ 
35:      if  $z = \text{FATAL}$  then
36:        return FINISH(FATAL,  $W$ )
37:      end if
38:      if  $z$  is a solution then
39:        if not VALIDATEANDPUBLISH( $z$ ) then
40:          return FINISH(FATAL,  $W$ )
41:        end if
42:        return FINISH(SUCCESS,  $W$ )
43:      end if
44:    end for
45:  end for
46:  return FINISH(NOTFOUND,  $W$ )
47: end function
```

▷ 候補 phase への一方方向遷移
▷ 全行の検証 pass、slot 0 を再利用

▷ 対角では同じ行を alias

命題 2 (最初の解の保存). 固定された入力に対し、参照実装と v2 提案実装の全精度演算がいずれも成功すると仮定する。両者が各行を $(N(v), i)$ 順に整列し、order 対、行内 index、合同解を同じ領域・順序で走査するならば、v2 提案実装は参照実装と同じ最初の解を返し、参照走査内に受理解がなければ両者とも NOTFOUND を返す。

証明. 命題 1 により、sketch 比較と exact replay で得る各行の順序は全精度ノルム行の順序と一致する。検証 pass は count 以外の状態を探索へ持ち込まず、探索 pass は各行を決定的に再生成する。対角時の $i_2 = i_1$ 開始、非対角時の $i_2 = 0$ 開始、 (j_1, j_2) の三角形順、およびアルゴリズム 3 の合同解列はいずれも参照走査と同じである。従って、最初に成功する tuple、または全 tuple の不成功が保存される。□

ここで FINISH は初期化済み固定精度 object を finalize した後、成功、失敗、未発見の別なく workspace 全体を `sqisign_secure_clear` する共通出口を表す。実装では long-lived lattice state が 77,168 bytes、phase union が 94,912 bytes であり、後者の candidate member は 14,024 bytes である。従って、`find_uv` の型サイズは

$$77,168 + \max(94,912, 14,024) = 172,080 \text{ bytes} \quad (8)$$

となる。

5.4 v2 提案実装の静的リンク閉包と消去

最終 firmware では、Level I/RADIX32 の KeyGen、Sign、Verify から到達する project translation unit を 52 個に固定した。ELF 監査により、allocator、GMP、system RNG、旧 large-stack 経路、未解決 symbol、内部 SRAM または XIP flash 外への allocated section が存在しないことを検査した。heap は 0 byte である。operation owner には前後 guard を置き、各操作後に workspace 全体が 0 であることを確認する。

この方法は、ソースに `malloc` が見えないことだけを根拠にしない。ライブラリ内部の allocator 依存、dead-strip されなかった旧経路、暗黙の VLA、linker script 上の未計上領域までを最終 ELF で検査対象に含める。

5.5 v3 適応実装の局所 lifetime scheduling

公式 v3 の `p324_3/m4f` を RP2350 向けにコンパイルすると、固定長として報告される最大の関数 frame は `quat_l1l_dual_reduce_ideal` の 34,816 bytes であった。この関数には、順に実行される処理段階の間に二つの非交差生存区間がある。

第一の重畳では、格子簡約状態 `fp_ldl_t` と、その簡約後に初めて構築する 4×4 整数行列 `Aout` を一つの型付き union へ置く。`fp_ldl_t` へのポインタは簡約ルーチンへ渡されるため、コンパイラが自動的に同じ stack slot を使うとは限らない。C の型配置で非交差性を明示することにより、この再利用を最終バイナリへ反映させる。

第二の重畳では、`ct_mk_result_t` 内の逆変換行列 `Ainv_total` から `Aout` を導出した後、その時点で不要となる `Ainv_total` の領域を基底出力用 `Bout` として再利用する。同じ構造体内の Gram 行列 `G` と変換 `T` は後続のノルム計算まで生存するため、これらは重畳しない。`Ainv_total` と `Bout` の領域サイズが等しいことは `_Static_assert` で検査する。

この二つの変更からなる構成が v3 適応実装である。アルゴリズム、分岐条件、乱数消費、公開 API は変更せず、局所変数の所有と配置だけを変更する。従って、v2 の certified norm sketch のようなデータ表現の変更ではなく、equations (1) and (3) の小さな実例である。

6 実装

6.1 v2 提案実装の構成と固定精度整数表現

実装は Compact Quaternion Algorithms の公開実装系列を出発点とし、SQIsign v2.0.1 Level I と、有限体・曲線層の RADIX32 backend へ固定した。RADIX32 は有限体 digit の選択であり、四元数整数の limb 幅ではない。四元数層の `ibz_t` は、1665-bit magnitude bound に符号 bit を加えて 64-bit limb 境界へ切り上げた `uint64_t`[27]、すなわち 1728-bit signed storage、1 整数当たり 216 bytes である。乗算、除算、GCD、平方根、modular exponentiation を含む多倍長演算は、この固定配列と専用 workspace 上で行う。有限体・曲線側も既存の固定幅実装を利用する。

この選択により、整数 object ごとの容量と alignment はコンパイル時に定まり、equations (1) and (3) の配置へ直接含められる。ただし固定幅であることは演算回数の固定を意味しない。現在の used-limb scan、比較、Euclid、指数演算は可変時間であり、本稿はこの整数 backend を定数時間と主張しない。v2 提案実装の SQIsign source commit は 71099e0827d3f0a3b3c705d2eda592c401e0d57d、RP2350 firmware commit は dc3289add3213cc7671f9943dfaa3bac770b2709 である。

6.2 v2 で GMP および mini-GMP を採用しない理由

GMP 6.3.0 の公式 manual では、`mpz_t` は size 情報と確保済み data への pointer を持ち、容量不足時には領域を追加確保し、既定では `malloc/realloc/free` を用いる [22]。従って通常の GMP は limb の確保・再確保を一般 heap へ委ね、allocator なしの bare-metal link closure と、コンパイル時に確定する排他的 SRAM 予約という本稿の要件を満たさない。mini-GMP は依存関係を小さくできるが、本実験で用いた bundled source も `mpz_t` ごとの可変長 limb 確保という所有モデルを保つため、stock 版をリンクするだけではこの問題を解かない。

不採用理由は正しさではない。公式 source commit dd133d7aca576c361a270c8e6434832535b42ecc の Level-I 実装だけを GMP backend 間で切り替えた別実験では、system GMP、native mini-GMP、secure static-pool 版 mini-GMP の全てが公式 100-vector KAT に合格した。一方、凍結 LP64 host corpus の Sign では、native mini-GMP の最大 requested live limb payload は 67,168 bytes であったが、最大 4,830 block が同時生存した。16-byte alignment の単純な secure first-fit allocator では、全 100 経路が通る最小 pool は 317,696 bytes であり、317,680 bytes では枯渇した。この値は allocator、ABI、入力 corpus に依存する測定閾値であって、mini-GMP 固有の下界でも Cortex-M33 上界でもない。また、この pool だけでは公式実装の stack、他の mutable state、MSP を計上していないため、v2 提案実装の 321,408 byte という firmware 全体の値とは直接比較しない。

性能も単純な優劣ではない。同じ公式 host source における native mini-GMP/system GMP の median 比は KeyGen で 1.3621、Sign で 1.4497 であった。clear-on-free を行う単純 pool は tracked mini-GMP に対し KeyGen 4.9441 倍、Sign 4.2316 倍を要したが、これは allocator prototype の cost であり mini-GMP 一般の cost ではない。個別演算では mini-GMP が GCD と逆元で速く、固定幅版が評価対象の乗算、除算、平方根で速かった。Arm GNU 15.2.1 の section GC 前 object text は、`-Os` で固定幅版 7,962 bytes、mini-GMP 側 18,566 bytes であった。従って本研究は速度支配を理由に固定幅版を選んだのではなく、次の global invariant を監査できることを理由に選んだ。

1. 全到達経路で heap を使用せず、operation reservation を link 前に上界化できる。
2. 数千の可変長 block に対する fragmentation、metadata、解放順の別証明を不要にする。
3. operation 終了時に owner 全体を一括消去し、解放済み limb の消去漏れを検査できる。
4. allocation size、count、再配置 address という追加の入力依存 trace を導入しない。

固定幅版にも値依存の制御・address trace が残るため、最後の項目は allocator 由来の漏洩面を除くという限定的な意味である。bounded mini-GMP と専用 region allocator は有効な別設計だが、全 retry 経路の pool bound、32-bit target 上の総 SRAM、消去、target cycle、漏洩 screen を同じ条件で示すまでは本稿の採用候補としない。

6.3 v2 提案実装のメモリ配置

v2 提案実装の操作 payload は 172,080 bytes、guard 込み operation owner は 172,208 bytes である。main SRAM bank には operation owner、131,072 byte の PSP 予約、その他 9,936 bytes を配置し、合計 313,216 bytes を link 時に使用する。別 scratch bank の 8,192 bytes を MSP 用に予約する。したがって排他的 SRAM 予約量は

$$313,216 + 8,192 = 321,408 \text{ bytes} \quad (9)$$

であり、211,072 bytes を未予約のまま残す。

6.4 v3 の構築条件と計上境界

v3 では、公式 source commit 6d017708db403bf83977fa70770fc4f7f9e9ff21 から生成した p324_3/m4f 実装を無改変の対照群とする。v3 適応実装は同じ commit から分岐し、src/quaternion/ref/lv1x/111/111_dim4.c だけを変更する。両系列とも、RP2350 が直接コンパイルする生成済み pqm4 source と、その生成元の追跡対象 source が byte 単位で一致することを SHA-256 digest で確認した。両者は Pico SDK 2.3.0、Arm GNU Toolchain 15.2.1、Release 構成、150 MHz、1 core という同じ条件でビルドする。

公式 v3 は、大きな局所状態を主として stack に置く。このため、v2 提案実装の operation arena payload と v3 の PSP 上書き深さは同じ量ではない。v3 firmware ではアプリケーション用の Process Stack Pointer (PSP) 領域を 131,072 bytes 予約し、各操作の直前に塗った pattern が連続して上書きされた深さを測る。割込み用の Main Stack Pointer (MSP) 領域は 8,192 bytes とする。

公式 v3 と v3 適応実装の linked heap section はいずれも 0 byte である。ただし、診断用の fprintf/abort 経路を介して newlib の allocator 関連 symbol が残り、GCC の stack-usage 出力には 19 件の動的 frame 記録がある。従って、v3 適応実装については成功した K/S/V 経路で heap 使用を観測しなかったことと、局所 frame を削減したことだけを主張する。v2 提案実装のように allocator symbol と動的 frame を link closure から全て除いたとは主張しない。

6.5 再現性のための固定条件

v2 実機試験では AES-256 CTR-DRBG の決定的 test adapter を使用し、host oracle と同じ seed、message、期待 digest を用いた。これは再現性と差分検査のためであり、製品向け entropy source ではない。clock、toolchain、linker layout、binary hash、capture hash を manifest へ固定した。KeyGen、Sign、Verify は同一 boot 内で直列に実行し、各操作後に guard と全消去を確認した。

v2 の host KAT では、保存済み NIST-v2 Level-I の公式 100-vector request (SHA-256 81ff60e3...d6ebc7e、完全値は補助資料) を、同一 commit から別々に構築した通常 D13 API と SQISIGN_LOWMEM_ONLY API へ与えた。低メモリ側は通常 entry を呼ばず、呼出側所有 workspace 版の KeyGen、Sign、Open、および Verify だけを用いる。各 vector で正当 signed message の復元、署名一 bit 改変の拒否、前後 guard、操作後消去を検査し、低メモリ run を新しい process で二回実施した。

v3 実機試験では、公式既知解テストの第 0 ベクトルを firmware へ埋め込み、公開鍵、秘密鍵、署名付き message、復元 message を byte 単位で照合した。無改変 v3 と v3 適応実装は別々の ELF、UF2、serial capture を持つ。時間評価は 5 組とし、奇数組では公式 v3、v3 適応実装、偶数組では v3 適応実装、公式 v3 の順に、それぞれ新しい boot で測定した。ホスト試験も別の build directory から実行する。v2 の source、build、result は移動も上書きもせず、v3 用の全成果物を専用 directory へ保存する。

7 評価方法

7.1 評価項目

評価項目を次の五群に分けた。

1. **機能同値性**: 参照経路との公開鍵、秘密鍵、signed message、post-RNG state の byte 一致、正当署名の受理、改変署名の拒否。
2. **メモリ**: 型の sizeof、link map、ELF section、heap policy、PSP/MSP watermark、未予約 SRAM。
3. **移植性**: RADIX64/RADIX32、Level I/III/V の関連 fixture、Arm compile、strict alias、VLA/alloca 禁止。
4. **性能**: host 上の交互順序反復と、RP2350 上の operation elapsed time。host と target の結果は混同しない。
5. **漏洩 screen**: 固定対ランダム タイミング検査、制御フロー・実効 address 列の再現性、局所 powmod

timing 診断。

7.2 版分離と比較設計

三つの実験系列、すなわち v2 提案実装、公式 v3、および v3 適応実装は、source、build、result の各領域を分離した。比較マニフェストは、各系列の commit、ELF、実機 capture、計測量の境界を SHA-256 digest で結び付ける。具体的なディレクトリ構成と digest は、GitHub 上の補助資料へ移す。

公式 v3 と v3 適応実装は同じ commit、parameter set、compiler option を使い、追跡対象の source 差分を一ファイルに限定する。この版内比較を局所重畳の効果推定に用いる。v2 と v3 の比較は、parameter、アルゴリズム、整数表現、メモリ所有モデルが同時に変わるため記述的な比較に限る。特に、v2 の明示的 arena と v3 の stack watermark を一つの「RAM 使用量」として減算しない。

7.3 正しさの gate

表 3 v2 提案実装で実施した主な acceptance gate

Gate	判定条件	結果
sketch collision	2^{100} と $2^{100} + 1$ が exact fallback を通り正順序	PASS
凍結トランスクリプト	PK/SK/signed message が 12 seed 全てで byte 一致	12/12
100-vector KAT	公式 request で通常 D13 と低メモリ D13 が byte 一致、独立二回	100/100 × 2
sanitizer	ASan および UBSan の対象 fixture が無警告完走	PASS
radix/level	RADIX64/RADIX32 および Level I/III/V の対象 gate	PASS
Arm ABI	candidate/phase/operation が規定 byte 数と一致	PASS
stack policy	選択済み閉包で VLA/alloca と dynamic stack record が 0	PASS
target K/S/V	同一 boot で KeyGen、Sign、Verify、guard、消去を完了	PASS
Verify RNG 非使用	Verify 前後で RNG state が不変	PASS

二つの低メモリ run と通常 D13 API が生成した三つの応答は全て byte 単位で一致し、SHA-256 は 1d86d15c5d2bfbef99f17634496086a3ab80f0e848d34d3e9409d75f3019dd68 であった。従って、観測した 100 入力では workspace 化による出力差を検出しなかった。ただし、Compact 固定精度 KeyGen は旧 GMP 実装と乱数消費量が異なるため、保存済み旧 NIST-v2 応答はこの比較の oracle ではない。本稿の KAT claim は、公式 request の全 100 入力を用いた同一 D13 algorithm 内の通常経路対 caller-owned 経路の一致であり、旧 GMP 応答との一致ではない。

v3 では公式 p324_3 の既知解を基準とし、無改変実装と v3 適応実装の双方へ同じ gate を適用した。表 4 の実機既知解テストは、単に自己生成した署名を自己検証する試験ではない。公式ベクトルの公開鍵、秘密鍵、署名付き message、復元 message との完全一致を要求する。

表 4 SQIsign v3.0 p324_3 の正しさ gate

Gate	v3 公式	v3 適応実装
ホスト NIST API 試験	PASS	PASS
ホスト自己試験	PASS	PASS
ホスト公式既知解テスト	PASS	PASS
RP2350 公式既知解テスト	PASS	PASS
改変署名の拒否	PASS	PASS

7.4 メモリ計測の区別

「operation arena payload」「guard 込み owner」「link 済み main bank」「PSP/MSP 予約」「観測 watermark」を別の量として記録する。watermark は一入力で実際に上書きされた深さであり、最悪 stack bound ではない。未使用領域を報告する際は、観測 watermark までを差し引くのではなく、保守的な予約全体を差し引く。

8 評価結果

8.1 候補領域と操作アリーナ

表 5 に全精度表実装から v2 提案実装への直接差分を示す。全精度ノルム二行を削除したことで candidate workspace は 14,024 bytes まで縮小した。しかし phase union は 94,912 bytes で止まる。これは既存の ML2 retry workspace が新たな最大 member になるためである。また、77,168 byte の lattice state は候補探索と同時生存するため、重畳していない。局所配列だけでなく干渉関係を扱わなければ、この支配項の移動は説明できない。

表 5 certified norm sketch による Cortex-M33/RADIX32 arena 差分

項目	全精度表 [B]	v2 提案 [B]	差分 [B]
常駐 full-norm 2 行	269,568	0	-269,568
candidate workspace	275,840	14,024	-261,816
phase union	275,840	94,912	-180,928
完全 operation arena	353,008	172,080	-180,928

完全 operation arena の削減率は

$$\frac{353,008 - 172,080}{353,008} = 0.51253229 \quad (10)$$

であり、51.2532% である。

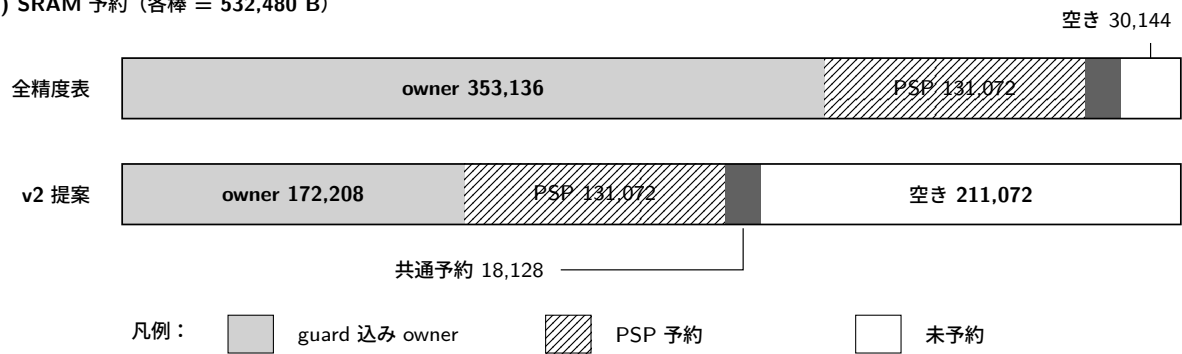
8.2 firmware 全体の SRAM と flash

表 6 に v2 提案実装の link 結果を示す。全精度表実装から v2 提案実装への 180,928 byte 削減は、そのまま排他的 SRAM 予約量と未予約 SRAM へ反映された。すなわち、単に heap を stack へ移した結果ではない。v2 提案実装の BIN は 288,384 bytes であり、全精度表実装より 880 bytes 大きい。これは再生成・比較コードの flash cost である。

表 6 v2 提案実装の RP2350 firmware メモリ内訳

項目	v2 提案 [B]	備考
オンチップ SRAM 総量	532,480	main+scratch
operation arena payload	172,080	KeyGen/Sign/Verify union
guard 込み operation owner	172,208	main bank
PSP 予約	131,072	main bank
その他 main-bank 領域	9,936	data/runtime 等
main-bank link 使用	313,216	上記の合計
MSP 予約	8,192	scratch bank
排他的 SRAM 予約	321,408	全精度表比 -180,928
未予約 SRAM	211,072	全精度表比 +180,928
heap	0	allocator symbol なし
.text	166,092	XIP flash
.rodata	116,232	XIP flash
.data	5,984	flash load/RAM
.bss	306,960	RAM、予約を含む
BIN image	288,384	全精度表比 +880

(a) SRAM 予約 (各棒 = 532,480 B)



(b) XIP BIN image

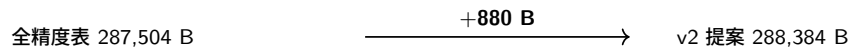


図 4 全精度表実装と v2 提案実装の RP2350 SRAM 予約。両棒はオンチップ SRAM 総量 532,480 bytes を同じ尺度で表す。owner は前後 guard 128 bytes を含む。「共通予約」18,128 bytes は MSP 8,192 bytes とその他 main-bank 領域 9,936 bytes の和である。PSP/MSP は watermark ではなく予約全体であり、白領域だけが未予約である。右下は BIN image の差分で、SRAM 棒には含まない。

8.3 実機 stack 観測

PSP には 131,072 bytes を予約し、一回の決定的実行で KeyGen 91,980 bytes、Sign 120,452 bytes、Verify 20,768 bytes の上書き extent を観測した。最も深い Sign でも観測上の余裕は 10,620 bytes であった。MSP は 8,192 byte 予約に対し保守的上限 2,396 bytes を観測した。

これらは pattern watermark であり、全入力・全割込み nest に対する証明ではない。そのため 211,072 byte という未予約量の算出では PSP/MSP の実測使用量ではなく予約全体を控除している。製品化時には割込み構成と production RNG を含む stack bound を再評価する必要がある。

8.4 実行時間

表 7 は一回の実機 boot で得た経過時間である。cycle 列は 150 MHz が操作中一定であるとして時間から換算した値で、performance counter の直接値ではない。KeyGen は約 45 分、Sign は約 127 分を要した。したがって本実装はメモリ実現可能性の証明であり、リアルタイム署名器ではない。

表 7 RP2350 上の一回の決定的 K/S/V 実行

操作	時間 [s]	150 MHz 換算 cycle	PSP extent [B]
KeyGen	2,698.150029	404,722,504,350	91,980
Sign	7,611.258527	1,141,688,779,050	120,452
Verify	0.814341	122,151,150	20,768

全精度表実装と v2 提案実装を交互順序で比較した host 試験は、15 seed、30 rows を二回行った。v2 提案実装と全精度表実装の median 比は KeyGen で 1.003695 と 1.006464、Sign で 0.999481 と 0.998980 であった。host 上では KeyGen に小さな sub-percent cost が見える一方、Sign slowdown の方向は再現しなかった。単一の target run 同士では KeyGen 1.000704、Sign 1.037312、Verify 1.000593 であったが、反復 target 分布がないため「Sign が一般に 3.7% 遅くなる」とは結論しない。

8.5 RAM-stack-flash-time の Pareto 解釈

v2 提案実装は全精度表実装に対し、operation RAM を大幅に減らし、局所 stack frame を 48–64 bytes 増やし、flash を 880 bytes 増やした。host KeyGen には小さな再計算 cost があるが、host Sign では増加を分離できていない。実機の絶対時間は長く、target 上の統計的な time cost は未確定である。このため v2 提案実装を「全指標で優れる」とは位置付けず、RAM を主要目的とした Pareto 点として扱う。

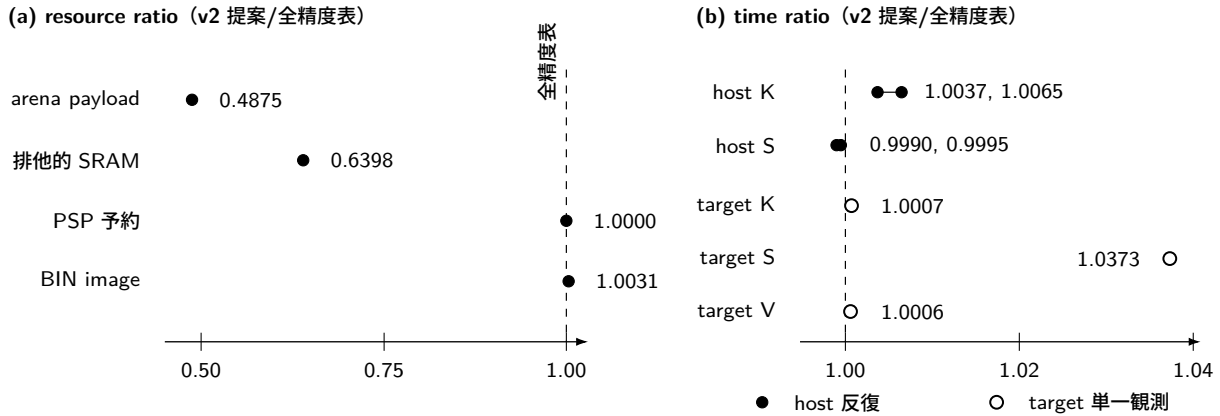


図 5 全精度表実装を 1 とした v2 提案実装の resource/time trade-off profile。resource 値は link・型から得た確定値である。host time は順序反転した二つの 30-row 実験を個別点で示し、target time は一回ずつの観測なので白抜き点とした。異なる測定条件の点は結んでいない。PSP 予約は不変だが、局所 frame は 48–64 bytes 増加した。

8.6 SQIsign v3 への転用追試

表 8 は、同じ公式 v3 commit から構築した無改変実装と v3 適応実装の版内比較である。無改変実装では、quat_l1l_dual_reduce_ideal の固定長 frame が 34,816 bytes であり、観測した Sign の PSP 上書き深さは 101,060 bytes であった。v3 適応実装は関数 frame を 30,888 bytes へ削減した。差分 3,928 bytes は、5 回全てで

実機 Sign の PSP 上書き深さの減少量と一致した。従って、この決定的ベクトルの反復では、変更した関数 frame が Sign の観測高水位に含まれ、二つの lifetime overlay が意図した物理 stack 削減として再現した。

表 8 RP2350 上の公式 SQIsign v3.0 と v3 適応実装の比較。時間列は各 5 回の中央値、差分列は 5 組の個別比率の中央値である。

項目	v3 公式	v3 適応実装	差分
最大固定 frame [B]	34,816	30,888	-3,928 (-11.282%)
KeyGen PSP extent [B]	56,972	56,972	0
Sign PSP extent [B]	101,060	97,132	-3,928 (-3.8868%)
Verify PSP extent [B]	37,444	37,444	0
crypto archive text [B]	204,157	203,945	-212 (-0.1038%)
KeyGen 時間 [s]	2.263611	2.269117	+0.2430%
Sign 時間 [s]	7.152943	7.182865	+0.4182%
Verify 時間 [s]	0.822095	0.820762	-0.1577%

KeyGen と Verify の PSP 上書き深さが変わらないことは、v3 適応実装が Sign の観測高水位だけを縮小したものと整合する。三操作の PSP 値は、公式版、v3 適応実装とも 5 回の版内反復で一定だった。linked crypto archive の text は 212 bytes 減少した。full ELF では実験識別用文字列も変わるため、コード差分の指標には同じ暗号 archive 境界を用いた。Sign 時間の対ごとの増加率は 0.4101–0.4226% で、平均 0.4170%、中央値 0.4182% だった。この狭い再現性は当該 binary、基板、clock、決定的 KAT ベクトルについての観測であり、入力分布や個体差を含む一般的な slowdown ではない。変更対象外の KeyGen と Verify にも小さな時間差があるため、code 配置を含む whole-image effect も排除できない。

版間の絶対時間も参考値として記録する。同じ RP2350 と 150 MHz で、v2 提案実装の KeyGen、Sign、Verify は一回の観測で 2,698.150029、7,611.258527、0.814341 s であり、v3 公式実装の 5 回中央値は 2.263611、7.152943、0.822095 s であった。v3 の KeyGen と Sign が大幅に短いことは明白だが、parameter、アルゴリズム、整数 backend、実装成熟度が同時に異なる。この差を一つの省メモリ変換による速度向上率とはしない。また、v2 提案実装は 172,080 byte の明示的 arena を PSP とは別に持つため、両版の PSP extent だけから総 SRAM の優劣を導かない。

9 サイドチャネル評価

本節の測定結果は v2 提案実装を対象とする。v3 適応実装について実施したのは機能同値性、stack、code size、経過時間の追試であり、固定対ランダム試験、制御フロー・address 差分、電力・電磁波取得はまだ行っていない。公式 v3 仕様も、提出実装全体は完全な定数時間実装ではないと明記する [5]。従って、v3 適応実装のメモリ削減や既知解一致を、サイドチャネル耐性へ外挿しない。

9.1 省メモリ化と定数時間性は独立である

固定されたアリーナ offset は allocator 由来の変動を除くが、演算回数や accessed value を秘密非依存にはしない。v2 提案実装には、sketch tie 回数、探索再生回数、可逆性判定、rejection、first-success publication などの入力依存処理が残る。さらに、元の SQIsign 署名経路には多倍長整数の used-limb scan、比較の early return、可変回数 Euclid、秘密由来指数を用いる modular exponentiation がある。

したがって、RAM 削減後には漏洩検査を再実行し、「メモリ配置が固定だから安全」という推論を採用しなかった。

9.2 固定対ランダム of timing screen

v2 提案実装について、固定対ランダム of 64 pair 試験を順序反転して二回実施した。一次統計 t_1 と、各 class で中心化した時間の平方へ同じ Welch 検定を適用する二次統計 t_2 を計算した結果を表 9 に示す。これは高次の非特定 t 検定で用いられる centered-moment 前処理に対応する [23]。一般的な探索閾値 $|t| > 4.5$ [23] に対し、両 run で t_2 が閾値を超えた。同一条件の negative control は安定し、run 間時間の Pearson 相関は 0.999929、Spearman 相関は 0.999771 であった。各 class 64 標本の小規模 screen であるため、閾値未満を非漏洩の証拠には用いない。一方、順序反転した二回で同方向の閾値超過が再現したことを、当該条件下の分布差を示す positive evidence として扱う。

表 9 v2 提案実装の Sign に対する固定対ランダム host timing screen

Run	t_1	t_2	判定
A	3.570	-5.410	二次 alert
B (順序反転)	3.576	-5.374	二次 alert 再現

sketch 比較回数、exact tie 回数、探索再生回数と Sign 時間の相関は、観測した指標では絶対 Pearson 値がおおむね 0.217 以下であった。従って、sketch が既存の支配的漏洩源であるとは示されていない。しかし相関が弱いことは非漏洩の証明でもなく、v2 提案実装をサイドチャネル改善と評価する根拠にはならない。

9.3 全 Sign の構造差分

別の二回の host 計測では、Sign 中の edge 列と対象 load/store の実効 address 列を記録した。同一 seed 同士の negative control は一致した一方、固定 seed と異なる signing-randomness の全比較で edge 列・address 列が再現可能に相違した。最初の差分は、多倍長整数の used-limb scan または most-significant-limb 比較を通り、random ideal sampler へ到達した。

この結果は、コンパイル済み全 Sign に秘密到達可能な可変制御があることを示す。ただし、変化させた直接入力 は署名乱数であり、最初の差分だけから長期秘密鍵が回復できるとは主張しない。

9.4 既報 SPA との対応

Mukherjee らは、Cornacchia 処理下の modular exponentiation で使われる秘密由来の一時指数を単純電力解析で回復し、秘密鍵回復へ接続する攻撃を報告した [13]。本実装の凍結 12-seed Sign corpus では、Sign → RepresentInteger → Cornacchia → sqrt_mod_p を 361 回観測し、そのうち 190 回が論文の指数関係に直接対応する modulus class であった。

RP2350 上の切り出し powmod timing 診断では、指数 Hamming weight と経過時間の Pearson 相関が 0.999860、回帰傾きが set bit 当たり約 4.004 ms となった。これは実チップ上の強い可変時間性を示すが、GPIO で区切った局所処理の時間計測であり、analog power/EM 波形または完全 Sign の鍵回復実験ではない。

9.5 対策 prototype と残課題

521 回固定の指数 schedule と mask selection を用いる第一 prototype は、同じ指数集合に対する Hamming-weight 傾きの絶対値を少なくとも 219.66 倍減らしたが、median 時間は 1.33872 倍となり、値依存時間が残った。乗算側も全 bit round を固定した第二 prototype は傾きを少なくとも 41,843 倍減らし、median は legacy の約 1.02426 倍であったが、固定・ランダム class 間に約 $-5 \mu\text{s}$ の再現差が残った。保護 sqrt を Sign 経路へ統合した host smoke は control の 9.447 倍を要した。

これらは、単一の可変 schedule を固定するだけでは、operand、Montgomery state、selector transition、

rejection、上位探索の漏洩が残ることを示す。現在の deployment 判定は不合格であり、完全 Sign の定数時間化、マスキング、二回以上の独立な power/EM acquisition、既報攻撃を positive control とする検出能力確認が必要である。

取得済み：positive evidence

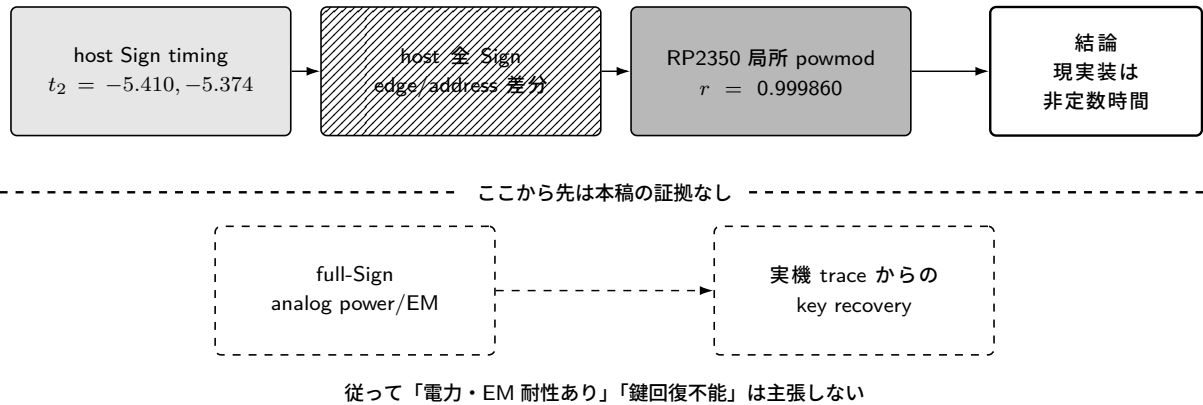


図6 本稿のサイドチャンネル証拠と主張境界。実線枠は取得済みの positive evidence、破線枠は未実施の実験を表す。取得済み結果から現実装が可変時間であることは言えるが、full-Sign の電力・EM 耐性や鍵回復可能性は判定できない。

10 考察

10.1 v2 から v3 へ転用できた要素

二つの版に共通して有効だったのは、暗号方式の特定の変数名ではなく、各オブジェクトの生存区間と干渉関係を明示し、型付き領域の配置へ反映する手順である。v2 提案実装は、この手順をプロトコル全体の operation owner へ適用した。v3 適応実装は、再設計後の別関数に同じ手順を適用し、最大固定 frame と実機 Sign watermark を同じ 3,928 bytes だけ削減した。この一致は、lifetime scheduling が v2 の `find_uv` だけに依存する工夫ではないことを示す一例である。

一方、certified norm sketch、1665-bit 整数幅、172,080 byte arena、および allocator/VLA を除去した 52 translation unit の link closure は v2 固有である。v3 には対象となる `find_uv` 候補表が存在せず、v3 適応実装には 19 件の動的 stack 記録が残る。このため、本稿は「v2 の配置を v3 へそのまま移植できた」とは結論しない。転用できたのは問題設定と検査手順であり、具体的な重畳対象と達成した保証は版ごとに異なる。

10.2 何が一般化できるか

本研究の個別定数は各版の Level I 相当 parameter と RP2350 に依存するが、方法は次の条件を持つ実装へ一般化できる可能性がある。

1. 大きな中間値を入力または小さな recipe から決定的に再生成できる。
2. 処理が phase へ分解でき、phase 間でポインタを保持しない API を設計できる。
3. パラメータごとに配列上界と alignment をコンパイル時に固定できる。
4. 代表値だけで順序を決められない場合に、厳密 fallback を許容できる。

特に certified sketch の本質は「上位 bit が異なる場合だけ比較を確定し、同じ場合は必ず原値へ戻る」ことである。ノルム以外にも、比較対象を小さな入力から厳密再構成できる探索・枝刈り処理へ適用できる。ただし、このデータ縮約を実証したのは v2 だけである。lifetime scheduling については v2 と v3 の二例を得たが、他 PQC

方式や他マイコンへの実証は行っていない。従って、SQIsign 以外への一般性は設計仮説の段階である。

10.3 自動化可能性

現在の layout は、手作業で定めた phase contract を C の型と複数の検査 script へ落としている。次段階では、各 workspace field へ `phase`, `size`, `align`, `secret`, `cleared-at` annotation を付け、次を自動化する余地がある。

1. call graph と phase state machine から干渉グラフを生成する。
2. alignment 制約付き offset 割当を解き、C の union/struct と static assertion を生成する。
3. 同一 offset を共有する field の生存区間非交差を検査する。
4. failure edge を含む消去義務と、秘密領域から公開出力への publication を検査する。
5. map/ELF と設計値を照合し、RAM-stack-flash の Pareto 表を再生成する。

ただし、C の pointer escape、alias、関数 pointer、LTO 後の call graph まで完全に扱う静的解析は容易ではない。当面は、annotation 駆動の layout generator と、既存の dynamic canary/zeroization fixture を組み合わせるのが現実的である。

10.4 未予約 SRAM の意味

v2 提案実装では 211,072 bytes が未予約となり、全精度表実装の 30,144 bytes に比べて実装余裕が大きくなった。この余裕は production RNG、通信 buffer、割込み stack、サイドチャネル対策用の追加状態、別 parameter set への拡張に利用できる可能性がある。ただし、未予約であることは、その全てを同時に収容できる証明ではない。特に定数時間・masking 実装は、追加 workspace と fresh randomness を必要とし得るため、メモリ最適化と安全性最適化を同一 Pareto 面で再評価すべきである。v3 については、v3 適応実装の局所 stack 削減だけを測定しており、同じ定義の排他的 SRAM 予約量と未予約 SRAM をまだ算出していない。

11 妥当性への脅威と限界

本結果には次の限界がある。

1. **単一 target・parameter:** 実機結果は RP2350、Arm Cortex-M33、1 core に限られる。v2 は Level I/RADIX32、v3 は p324_3/m4f だけを評価した。Level III/V および他 MCU の fit を示さない。
2. **決定的乱数:** v2 は再現用 CTR-DRBG adapter、v3 は公式既知解ベクトルを用いる。entropy source、reseeding、故障時処理を含む production RNG 統合は未評価である。
3. **実行回数:** v2 の target 時間と stack watermark は一回の決定的 K/S/V である。v3 は順序を反転した 5 組を測定したが、単一基板・単一 KAT ベクトルであり、入力分布、個体差、worst-case stack、retry tail を与えない。
4. **機能同値性の範囲:** v2 の 12 個の凍結トランスクリプト、公式 request を用いた二回の 100-vector 通常・低メモリ一致、および v3 の公式既知解一致は強い回帰 gate だが、有限 corpus であり、全入力の形式的 equivalence proof ではない。また v2 KAT の oracle は同一 commit の通常 D13 API であり、乱数消費列が異なる旧 GMP 応答ではない。
5. **手動 lifetime:** 生存区間の定義は人手の phase contract に依存する。sanitizer、canary、ELF 監査、公式既知解は誤りを検出するが、完全性を証明しない。
6. **v3 の静的閉包:** v3 適応実装には 19 件の動的 stack 記録と診断経路由来の allocator 関連 symbol が残る。v2 提案実装と同じ no-VLA/no-allocator closure ではなく、最悪 stack bound も与えない。
7. **可変時間:** v2 署名には入力依存の分岐、address、retry、再計算が残る。完全 Sign の analog power/EM test

と key-recovery reproduction は未実施である。v3 適応実装のサイドチャネル測定も未実施である。

8. **版の固定:** v2 結果は source commit 71099e08...、v3 結果は 6d017708... に固定される。今後の仕様・実装変更には再評価が必要である。
9. **実用性能:** v2 提案実装の KeyGen と Sign は非常に長く、省メモリ化だけでは製品利用に十分でない。v3 は大幅に高速だが、v3 適応実装の時間評価は単一 KAT ベクトルの反復に限られ、製品向け乱数も評価していない。

12 今後の研究工程

短期的には、論文文化に必要な残りの証拠強度を上げる。v2 提案実装専用の 100-vector host KAT gate は凍結済みである。次に、v2 の複数回 target 計測、v3 適応実装の複数入力・複数 binary 配置での反復、stack bound の静的・動的交差検査、図表値の artifact からの自動生成、投稿直前の文献再監査を行う。v3 については 19 件の動的 stack 記録を一件ずつ上界化または除去し、診断経路を含む allocator symbol policy を定める。これにより、v3 の単一ベクトル反復を入力分布へ拡張し、観測 watermark を静的上界で補強する。

中期的には、lifetime annotation から干渉グラフ、arena layout、static assertion、消去 check を生成する tool を実装する。本稿で手作業により適用した一連の変換を入力データとして、生成 layout が既存 172,080 byte 点を再現できるかを最初の評価課題とする。

安全性については、メモリ論文と並行しても claim を混ぜない。v2 では random ideal sampler、Cornacchia、固定幅整数、格子・四元数処理を秘密非依存の固定 work へ移し、全 Sign edge/address negative result、RP2350 timing negative result、既報攻撃を検出できる analog positive control、独立二回の power/EM fixed-vs-random 試験を順に通す。v3 適応実装には同じ順序で独立の漏洩評価を行う。各対策は、追加 RAM、stack、flash、時間を版ごとの Pareto 形式で報告する。

他 PQC 方式や複数マイコンへの展開は、上記の自動化と単一 target 上の証拠を固めた後の課題とする。本稿の初版では一般化可能性を議論するに留め、未測定の移植性を主貢献として主張しない。

13 結論

本稿では、SQIsign v2.0.1 Level I の完全な KeyGen、Sign、Verify 経路を、RP2350 のオンチップ SRAM 内で GMP および動的確保なしに実行するためのメモリ設計を示した。型付き lifetime-aware arena scheduling により、個別関数の配列サイズではなくプロトコル全体の同時生存領域を削減した。さらに、候補ノルムをビット長と上位 64 bit へ縮約し、同値時に厳密再生成する certified norm sketch を導入した。exact fallback により元の全精度順序を保ったまま、operation arena を 353,008 bytes から 172,080 bytes へ削減した。

v3.0 では、v2 固有の候補表が消えた後も、同じ生存区間の考え方を格子簡約 frame へ適用した。v3 適応実装は公式既知解との一致を保ち、Sign の PSP 上書き深さを 101,060 bytes から 97,132 bytes へ削減した。この追試は、lifetime scheduling の転用可能性を一例で支持する。同時に、certified norm sketch と v2 の静的 closure はそのまま転用できないことも示した。

v2 最終 firmware は 321,408 bytes を排他的に予約し、211,072 bytes を未予約として残しながら、決定的 K/S/V を一回の boot で完了した。host では公式 request を用いる 100-vector KAT を二回走完し、同一 commit の通常 D13 API との byte 一致、正当 message 復元、改変拒否、arena 境界・消去を各 100 本で確認した。一方、v2 の KeyGen と Sign は非常に遅く、署名の可変時間性と既知の SPA 攻撃面も残る。v3 適応実装にも動的 stack と未評価の漏洩面が残る。従って本結果の価値は、値の上界、同時生存、再計算、link 時予約、安全性の主張範囲を版ごとに分離し、二世代の実装で検証可能な設計として示した点にある。

成果物と再現性

ソースコード、再構成用パッチ、実験スクリプト、バイナリの digest、および実機 capture を含む補助資料は、<https://github.com/quantumshiro/tinysqisign> で公開する。

参考文献

- [1] G. Alagic, M. Bros, P. Ciadoux, Q. Dang, T. H. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, H. Silberg, D. Smith-Tone, and N. Waller. *Status Report on the Second Round of the Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process*. Tech. rep. NIST IR 8610. National Institute of Standards and Technology, May 2026. DOI: 10.6028/NIST.IR.8610. URL: <https://doi.org/10.6028/NIST.IR.8610>.
- [2] National Institute of Standards and Technology. *Round 3 Additional Digital Signature Schemes*. 2026. URL: <https://csrc.nist.gov/projects/pqc-dig-sig/round-3-additional-signatures> (visited on 09/03/2026).
- [3] L. De Feo, D. Kohel, A. Leroux, C. Petit, and B. Wesolowski. “SQISign: Compact Post-Quantum Signatures from Quaternions and Isogenies”. In: *Advances in Cryptology – ASIACRYPT 2020*. Vol. 12491. Lecture Notes in Computer Science. Springer, 2020, pp. 64–93. DOI: 10.1007/978-3-030-64837-4_3. URL: https://doi.org/10.1007/978-3-030-64837-4_3.
- [4] SQISign submission team. *SQISign: Algorithm Specifications and Supporting Documentation, Version 2.0.1*. July 2025. URL: <https://sqisign.org/spec/sqisign-20250707.pdf> (visited on 09/03/2026).
- [5] SQISign submission team. *SQISign: Algorithm Specifications and Supporting Documentation, Version 3.0*. Tech. rep. National Institute of Standards and Technology, Sept. 2026. URL: <https://sqisign.org/spec/sqisign-20260901.pdf> (visited on 09/03/2026).
- [6] W. Kim, J. Lee, H. Kim, and C. Lee. “SQISign with Fixed-Precision Integer Arithmetic”. In: *Cryptology ePrint Archive, Paper 2025/1649* (2025). URL: <https://eprint.iacr.org/2025/1649>.
- [7] W. Kim, C. Lee, and H. Yoo. “Compact Quaternion Algorithms for SQISign”. In: *Cryptology ePrint Archive, Paper 2026/1031* (2026). URL: <https://eprint.iacr.org/2026/1031>.
- [8] M. J. Kannwischer, M. Krausz, R. Petri, and S.-Y. Yang. “pqm4: Benchmarking NIST Additional Post-Quantum Signature Schemes on Microcontrollers”. In: *Cryptology ePrint Archive, Paper 2024/112* (2024). URL: <https://eprint.iacr.org/2024/112>.
- [9] F. Carvalho Rodrigues, D. Gazzoni Filho, G. Adj, I. A. Canales-Martínez, J. Chávez-Saab, J. López, M. Scott, and F. Rodríguez-Henríquez. “Generation of Fast Finite Field Arithmetic for Cortex-M4 with ECDH and SQISign Applications”. In: *Cryptology ePrint Archive, Paper 2025/1322* (2025). URL: <https://eprint.iacr.org/2025/1322>.
- [10] M. A. Aardal, G. Adj, A. Alblooshi, D. F. Aranha, I. A. Canales-Martínez, J. Chávez-Saab, D. L. Gazzoni Filho, K. Reijnders, and F. Rodríguez-Henríquez. “Optimized One-Dimensional SQISign Verification on Intel and Cortex-M4”. In: *Cryptology ePrint Archive, Paper 2024/1563* (2024). URL: <https://eprint.iacr.org/2024/1563>.
- [11] L. De Feo, L.-J. Jian, T.-Y. Wang, and B.-Y. Yang. “SQISign on ARM”. In: *Cryptology ePrint Archive, Paper 2026/394* (2026). URL: <https://eprint.iacr.org/2026/394>.
- [12] SQISign submission team. *The SQISign Implementation, Version 3.0*. Git tag `nist-v3`, commit `6d017708db403bf83977fa70770fc4f7f9e9ff21`. Sept. 2026. URL: <https://github.com/SQISign/the-sqisign/tree/nist-v3> (visited on 09/03/2026).

- [13] A. Mukherjee, M. Czaprynsko, D. Jacquemin, P. Kutas, and S. Sinha Roy. “Simple Power Analysis Attack on SQIsign”. In: *Cryptology ePrint Archive, Paper 2025/830* (2025). URL: <https://eprint.iacr.org/2025/830>.
- [14] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein. “Register Allocation via Coloring”. In: *Computer Languages* 6.1 (1981), pp. 47–57. DOI: 10.1016/0096-0551(81)90048-5.
- [15] J. Gergov. “Algorithms for Compile-Time Memory Optimization”. In: *Proceedings of the Tenth Annual ACM-SIAM Symposium on Discrete Algorithms*. ACM/SIAM, 1999, pp. 907–908. DOI: 10.5555/314500.315082.
- [16] G. Borin, M. Corte-Real Santos, J. Komada Eriksen, R. Invernizzi, M. Mula, S. Schaeffler, and F. Vercauteren. “Qlapoti: Simple and Efficient Translation of Quaternion Ideals to Isogenies”. In: *Cryptology ePrint Archive, Paper 2025/1604* (2025). URL: <https://eprint.iacr.org/2025/1604>.
- [17] Y.-F. Lai. “Qlapoti+ and More: Optimizing Isogeny-based Signatures”. In: *Cryptology ePrint Archive, Paper 2026/1640* (2026). URL: <https://eprint.iacr.org/2026/1640>.
- [18] M. Duparc, A. Leroux, and S. Schaeffler. “Qlapoty: Improved Analysis and Efficiency for Quaternionic Ideal to Isogeny Transformation”. In: *Cryptology ePrint Archive, Paper 2026/1700* (2026). URL: <https://eprint.iacr.org/2026/1700>.
- [19] F. Kouider, A. Mukherjee, D. Jacquemin, and P. Kutas. “Constant-time Integer Arithmetic for SQIsign”. In: *Cryptology ePrint Archive, Paper 2025/832* (2025). URL: <https://eprint.iacr.org/2025/832>.
- [20] O. Hanyecz, A. Karenin, E. Kirshanova, P. Kutas, and S. Schaeffler. “Constant Time Lattice Reduction in Dimension 4 with Application to SQIsign”. In: *Cryptology ePrint Archive, Paper 2025/027* (2025). URL: <https://eprint.iacr.org/2025/027>.
- [21] A. Basso, C. He, D. Jacquemin, F. Kouider, P. Kutas, A. Mukherjee, S. Schaeffler, and S. Sinha Roy. “Constant-time Quaternion Algorithms for SQIsign”. In: *Cryptology ePrint Archive, Paper 2025/2192* (2025). URL: <https://eprint.iacr.org/2025/2192>.
- [22] T. Granlund and the GMP development team. *GNU MP: The GNU Multiple Precision Arithmetic Library*. 6.3.0. 2023. URL: <https://gmplib.org/gmp-man-6.3.0.pdf> (visited on 09/03/2026).
- [23] T. Schneider and A. Moradi. “Leakage Assessment Methodology: A Clear Roadmap for Side-Channel Evaluations”. In: *Journal of Cryptographic Engineering* 6.2 (2016), pp. 85–99. DOI: 10.1007/s13389-016-0120-y. URL: <https://eprint.iacr.org/2015/207>.